Lab 1 Matlab 基本語法與矩陣運算

基本運算規則

(1) Format and precision aer critical.

```
8/3
```

```
ans = 2.6667
```

```
int8(8)/int8(3)
```

```
ans = int8

    3
```

(2) Variable/file name can not start with number.

```
A1=10
```

```
A1 = 10
```

(3) Do not use preserved variable/function name.

pi, inf,nan, sin, cos, max, min, sum, for, end, ...

```
pi
```

```
ans = 3.1416
```

```
pi=2
```

```
pi = 2
```

```
pi
```

```
pi = 2
```

常用指令

(1) clc %清除command window

(2) clear all %清除workspace中所有儲存變數

(3) close all %清除所有以開啟視窗

(4) % %註解

(5) ; %換行符號/不輸出結果至command window

(6) = % 指派等號右邊數值給左邊變數

Mathematical operators

Mathematical notation	Matlab-command	Mathematical notation	Matlab-comma
a+b	a+b	i	i
a-b	a-b	$4 \cdot 10^3$	4e3 or 4*10^3
ab	a*b	$3-4i$	3-4*i or 3-4*j
3xy	3*x*y	sin x, arctan x, ...	sin(x), atan(x), ..
$\frac{a}{b}$	a/b or a\b	e^x	exp(x)
a^b	a^b	ln x	log(x)
$\sqrt{x}$	sqrt(x) or x^0.5	log x	log10(x)
π	pi	|x|	abs(x)
e	exp(1)		

Mathematical functions

abs(x)	絕對值	median(x)	中位數
sin(x)	正弦值	sinh(x)	雙曲正弦值
exp(x)	自然指數	asin(x)	反正弦值
log(x)	自然對數	inv(x)	反矩陣
sqrt(x)	開平方根	rank(x)	秩
min(x)	最小值	round(x)	四捨五入至整數
max(x)	最大值	floor(x)	無條件捨去至整
mean(x)	平均值	ceil(x)	無條件進入至整
sort(x)	排序	sum(x)	總和
std(x)	標準差		

符號運算

Example: 求一元二次方程式 $ax^2+bx+c=0$ 的解公式。

```
syms a b c x
x=solve(a*x^2+b*x+c==0, x)
```

x =

$$\begin{pmatrix} -\frac{b+\sqrt{b^2-4\,a\,c}}{2\,a} \\ -\frac{b-\sqrt{b^2-4\,a\,c}}{2\,a} \end{pmatrix}$$

Variable definition

(1) The 1st character must be a letter.

(2) The variable name is compopsed of letters, numbers, and underscore (_).

(3) The maximum length of the variable neme is 63 characters.

(4) Use "=" to assign right-hand side value to the left-hand side variable.

Data type of variable

(1) Numbers

```
a=1974
```

```
a = 1974
```

(2) Letter/Striing

```
b='x'
```

```
b =
'x'
```

```
c='this is a string.'
```

```
c =
'this is a string.'
```

(3) Array/Matrix

```
A=[1 2 3 4 5 6]
```

```
A = 1×6
     1     2     3     4     5     6
```

```
A=[1,2,3,4,5,6]
```

```
A = 1×6
     1     2     3     4     5     6
```

```
A=[1 2 3; 4 5 6]
```

```
A = 2×3
     1     2     3
     4     5     6
```

```
A=[1:10]
```

```
A = 1×10
     1     2     3     4     5     6     7     8     9    10
```

```
A=[1:10; 11:20]
```

```
A = 2×10
     1     2     3     4     5     6     7     8     9    10
    11    12    13    14    15    16    17    18    19    20
```

Data Precision

(1) double : convert to 64-bit floating point

(2) single : convert to 32-bit floating point

(3) uint8 : convert to unsigned 8-bit integrer

(4) uint16 : convert to unsigned 16-bit integrer

(5) uint32 : convert to unsigned 32-bit integrer

(6) uint64 : convert to unsigned 64-bit integrer

(7) int8 : convert to signed 8-bit integrer

(8) int16 : convert to signed 16-bit integrer

(9) int32 : convert to signed 32-bit integrer

(10) int64 : convert to signed 64-bit integrer

(11) logical : convert numeric value to logical (8-bit)

(12) char : convert nonnegative integer array into a character array (16 -bit)

```
a=1;
b=int8(a);
whos
```

```
  Name      Size            Bytes  Class     Attributes

  A         2x10              160  double
  A1        1x1                 8  double
  a         1x1                 8  double
  ans       1x1                 8  double
  b         1x1                 1  int8
  c         1x17               34  char
  col       0x1                 0  double
  pi        1x1                 8  double
  row       0x1                 0  double
  x         2x1                 8  sym
```

```
A=ones(10000, 10000);
B=int8(A);
whos
```

```
  Name          Size                    Bytes  Class     Attributes

  A         10000x10000            800000000  double
```

```
  A1            1x1                        8  double
  B         10000x10000            100000000  int8
  a             1x1                        8  double
  ans           1x1                        8  double
  b             1x1                        1  int8
  c             1x17                      34  char
  col           0x1                        0  double
  pi            1x1                        8  double
  row           0x1                        0  double
  x             2x1                        8  sym
```

Check capacity of data type

(1) intmax, intmin

```
intmax('int32')
```

```
ans = int32

    2147483647
```

```
intmin('int8')
```

```
ans = int8

    -128
```

(2) realmax, realmin

```
realmax('double')
```

```
ans = 1.7977e+308
```

Check ASCII code and UNICODE

```
abs('a')
```

```
ans = 97
```

```
char(97)
```

```
ans =
'a'
```

```
abs('台灣科技大學')
```

```
ans = 1×6
       21488       28771       31185       25216       22823       23416
```

```
char([21488 28771 31185 25216 22823 23416])
```

```
ans =
'台灣科技大學'
```

```
str2num('97')
```

```
ans = 97
```

```
num2str([21488 28771 31185 25216 22823 23416])
```

```
ans =
'21488  28771  31185  25216  22823  23416'
```

Index of matrix

(1) MATLAB是MATrix LABoratory的縮寫。

(2) Column-wise index

(1)	(6)	(11)	(16)	(21)
(2)	(7)	(12)	(17)	(22)
(3)	(8)	(13)	(18)	(23)
(4)	(9)	(14)	(19)	(24)
(5)	(10)	(15)	(20)	(25)

(3) Row-column index

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
(2,1)	(2,2)	(2,3)	(2,4)	(2,5)
(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
(4,1)	(4,2)	(4,3)	(4,4)	(4,5)
(5,1)	(5,2)	(5,3)	(5,4)	(5,5)

```
A=[1 2 3 4 5; 6 7 8 9 10; 11 12 13 14 15; 16 17 18 19 20; 21 22 23 24 25]
```

```
A = 5×5
     1     2     3     4     5
     6     7     8     9    10
    11    12    13    14    15
    16    17    18    19    20
    21    22    23    24    25
```

```
A(18)
```

```
ans = 14
```

```
A(3,4)
```

```
ans = 14
```

Get values from the matrix

Column-wise index

(1)	(6)	(11)	(16)	(21)
(2)	(7)	(12)	(17)	(22)
(3)	(8)	(13)	(18)	(23)
(4)	(9)	(14)	(19)	(24)
(5)	(10)	(15)	(20)	(25)

Row-column index

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
(2,1)	(2,2)	(2,3)	(2,4)	(2,5)
(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
(4,1)	(4,2)	(4,3)	(4,4)	(4,5)
(5,1)	(5,2)	(5,3)	(5,4)	(5,5)

```
A(6:10)
```

```
ans = 1×5
     2     7    12    17    22
```

```
A([12:14 17:19])
```

```
ans = 1×6
     8    13    18     9    14    19
```

```
A(:,2)
```

```
ans = 5×1
     2
     7
    12
    17
    22
```

```
A(2:4, 3:4)
```

```
ans = 3×2
     8     9
    13    14
    18    19
```

Delete rows or columns

Row-column index

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
(2,1)	(2,2)	(2,3)	(2,4)	(2,5)
(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
(4,1)	(4,2)	(4,3)	(4,4)	(4,5)
(5,1)	(5,2)	(5,3)	(5,4)	(5,5)

```
A(4,:)=[]
```

```
A = 4×5
     1     2     3     4     5
     6     7     8     9    10
    11    12    13    14    15
    21    22    23    24    25
```

```
A=[1 2 3 4 5; 6 7 8 9 10; 11 12 13 14 15; 16 17 18 19 20; 21 22 23 24 25];
[row, col]=find(A==13)
```

```
row = 3
col = 3
```

% Relation operators

Equal	==
Not equal	~=
Less than	<
Greater than	>
Less than or equal	<=
Greater than or equal	>=

% logical operators

And	&
Or	|
Not	~

Set values

Row-column index

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
(2,1)	(2,2)	(2,3)	(2,4)	(2,5)
(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
(4,1)	(4,2)	(4,3)	(4,4)	(4,5)
(5,1)	(5,2)	(5,3)	(5,4)	(5,5)

```
A(2:4, 3:4)=100
```

```
A = 5×5
     1     2     3     4     5
     6     7   100   100    10
```

```
    11    12   100   100    15
    16    17   100   100    20
    21    22    23    24    25
```

```
A(2:4, 3:4)=[100 101; 102 103; 104 105]
```

```
A = 5×5
     1     2     3     4     5
     6     7   100   101    10
    11    12   102   103    15
    16    17   104   105    20
    21    22    23    24    25
```

Matrix generator

(1) ones : create a "ones" array

```
ones(3,4)
```

```
ans = 3×4
     1     1     1     1
     1     1     1     1
     1     1     1     1
```

(2) zeros : create a "zeros" array

```
zeros(3)
```

```
ans = 3×3
     0     0     0
     0     0     0
     0     0     0
```

(3) rand : create an array with "uniformly" distributed pseudorandom numbers

```
rand(3)
```

```
ans = 3×3
    0.5005    0.6820    0.5216
    0.4711    0.0424    0.0967
    0.0596    0.0714    0.8181
```

(4) randn : create an array with "normally" distributed pseudorandom numbers

```
randn(3)
```

```
ans = 3×3
    0.6770    0.4494    0.5362
    0.8577    0.1006    0.8979
   -0.6912    0.8261   -0.1319
```

Useful function

(1) size : size of array

```
size(A)
```

```
ans = 1×2
     5     5
```

(2) length : length of vector

```
V=ones(1, 45);
size(V)
```

```
ans = 1×2
     1    45
```

```
length(V)
```

```
ans = 45
```

Matrix operation

(1) A+B or A-B : Matrices A and B must have same dimension.

(2) A*B : inner dimensions must agree.

(3) A.*B or A./B : Matrices A and B must have same dimension.

```
A=6*ones(3)
```

```
A = 3×3
     6     6     6
     6     6     6
     6     6     6
```

```
B=2*ones(3)
```

```
B = 3×3
     2     2     2
     2     2     2
     2     2     2
```

```
A*B
```

```
ans = 3×3
    36    36    36
    36    36    36
    36    36    36
```

```
A.*B
```

```
ans = 3×3
    12    12    12
    12    12    12
    12    12    12
```

(4) A^k : Matrix A must be a squre matrix.

(5) A.^k : No specific requirement for matrix dimension.

```
A^2
```

```
ans = 3×3
   108   108   108
   108   108   108
   108   108   108
```

```
A*A
```

```
ans = 3×3
   108   108   108
   108   108   108
   108   108   108
```

```
A.^2
```

```
ans = 3×3
    36    36    36
    36    36    36
    36    36    36
```

(6) A/B or A*inv(B) : Inner matrix dimensions of A and inverse matrix of B must agree.

(7) A\B or inv(A)*B : Inner matrix dimensions of inverse matrix of A and B must agree.

Example: 解聯立方程式

$$\begin{cases} 3x + 4y = 10 \\ 5x - 2y = 8 \end{cases}$$

$$\begin{bmatrix} 3 & 4 \\ 5 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 10 \\ 8 \end{bmatrix}$$

```
A=[3 4; 5 -2];
B=[10; 8];
X=A\B
```

```
X = 2×1
     2
     1
```

```
X=inv(A)*B
```

```
X = 2×1
     2
     1
```

(8) A' : matrix transpose

```
A=zeros(3);
A(:)=[1:9]
```

```
A = 3×3
     1     4     7
     2     5     8
     3     6     9
```

```
A'
```

```
ans = 3×3
     1     2     3
     4     5     6
     7     8     9
```